

6. Гілки, Pull Request, Merge та оновлення проєкту

Робота з гілками (branches)

Гілки — це паралельні «лінії розвитку» проєкту. `main` завжди залишається робочою й стабільною, а нову функцію робимо в окремій гілці.

```
main      A---B---C-----F      (merge)
              \       /
feature-login  D---E---
```

Створення і перемикання

```
git branch feature-login      # створити гілку
git checkout feature-login    # перейти на неї
# або коротше – створити і одразу перейти
git checkout -b feature-login
```

Сучасний варіант тих самих команд (Git 2.23+):

```
git switch feature-login      # перейти
git switch -c feature-login    # створити і перейти
```

Перевір, на якій гілці ти:

```
git branch      # * стоїть біля поточної
git branch -a   # разом з віддаленими
git status      # перший рядок – On branch ...
```

Після змін

```
git add .
git commit -m "add login feature"
git push origin feature-login
```

`-u` при першому пуші гілки (`git push -u origin feature-login`) дозволить далі писати просто `git push`.

Назви гілок — прийнята практика

- `feature/login-form` — нова функція;
- `fix/division-by-zero` — виправлення;
- `hotfix/payment` — термінове виправлення на проді;
- без пробілів і кирилиці, слова через `-`.

Видалення гілки

```
git branch -d feature-login      # локально (після злиття)
git branch -D feature-login      # локально, навіть якщо не злита
git push origin --delete feature-login # на сервері
```

Pull Request (PR) — командна магія

На GitHub:

1. відкрий свою гілку → **Compare & pull request**;
2. напиши короткий опис змін;
3. додай рев'юера (того, хто перевірятиме код);
4. натисни **Create pull request**.

PR — це пропозиція злити твої зміни в основну гілку. Команда перевіряє → коментує → після схвалення → **Merge**.

У GitLab це називається **Merge Request (MR)** — суть та сама.

Що робить PR корисним:

- назва по суті («Add login form», не «правки»);
- в описі — що змінилось і як перевірити;
- маленький обсяг: 300 рядків рев'юють уважно, 3000 — ні.

Merge — злиття змін

Після схвалення PR обираємо тип злиття:

Тип злиття	Опис
Merge commit	створює додатковий коміт для об'єднання
Squash merge	об'єднує всі зміни в один коміт
Rebase merge	робить історію «чистою» без комітів об'єднання

На початку — найпростіше користуватись **Merge commit**.

Те саме можна зробити локально:

```
git checkout main          # перейти в гілку, КУДИ зливаємо
git merge feature-login    # злити гілку, ЗВІДКИ беремо зміни
```

Оновлення проєкту

Отримати нові зміни:

```
git pull origin main
```

Перевірити без злиття (тільки метадані):

```
git fetch
git log origin/main --oneline    # подивитися, що там нового
```

Різниця проста: `git fetch` — «дізнатися, що змінилось», `git pull` — «дізнатися і одразу влити до себе». Фактично `git pull` = `git fetch` + `git merge`.

Правило команди: **починай робочий день з `git pull`**, інакше твоя гілка відстає й на push буде відмова.

Конфлікти злиття

Конфлікт виникає, коли ти і колега змінили **ті самі рядки** того самого файлу. Git не вгадує, чия правка правильна, і просить вирішити вручну:

```
<<<<<<< HEAD
print("Привіт з main")
=====
print("Привіт з feature-login")
>>>>>> feature-login
```

Що робити:

1. відкрити файл, залишити правильний варіант, видалити маркери `<<<<<<<`, `=====`, `>>>>>>`;
2. `git add <file>` — так ти кажеш Git «конфлікт вирішено»;
3. `git commit` — завершити злиття.

Приклад із двома змінами того самого рядка:

```
<<<<<< HEAD
print("Hello world")
=====
print("Hello Git!")
>>>>>> feature-branch
```

Вибери потрібну версію, збережи файл і зроби коміт:

```
git add file.py
git commit -m "resolve conflict"
```

Скасувати злиття і повернутись, як було:

```
git merge --abort
```

Конфлікт — це нормальна робоча ситуація, а не поломка. Найкраща профілактика: маленькі гілки, частий `git pull`, різні люди — різні файли.

Збереження тимчасових змін (`git stash`)

Якщо треба швидко переключитися на іншу гілку, але не хочеш комітити недороблене:

```
git stash          # відкласти зміни (робоча тека стає чистою)
git stash list     # переглянути відкладене
git stash pop      # повернути останнє і прибрати зі списку
```

Ще кілька корисних варіантів:

```
git stash -u          # разом з новими (untracked) файлами
git stash save "half of form" # з підписом, щоб потім не гадати
git stash apply       # повернути, але лишити копію у списку
git stash drop        # викинути останнє відкладене
git stash clear       # очистити все
```

`stash` — це «кишеня», а не історія: там немає комітів, і легко забути, що щось лежить. Не тримай зміни в `stash` днями — або комітити, або викидати.